

EN3160 Assignment 01

Name: Wedamestrige A.N.

Index Number: 230687P

GitHub Repository: <https://github.com/AkhilaNisal/EN3160Assignment1>

Task 1 - Piecewise-linear intensity transformation

Brief description: Apply the discontinuous transformation in Fig. 1a to the Emma image, then compare several smooth, custom piecewise-linear curves. Discontinuities can create abrupt visible transitions, so a smooth curve is selected for the final custom result.

Chosen custom curve: Custom 2, because it gives a noticeable but natural contrast increase without the sharp intensity jumps of Fig. 1a.

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt

def build_lut(breakpoints):
    """Build an 8-bit piecewise-linear lookup table; repeated x values make jump
    bp = np.asarray(breakpoints, dtype=np.float32)
    lut = np.zeros(256, dtype=np.float32)
    for i in range(len(bp) - 1):
        x0, y0 = bp[i]
        x1, y1 = bp[i + 1]
        if x1 == x0:
            lut[int(x1)] = y1
            continue
        xs = np.arange(int(x0), int(x1) + 1)
        lut[xs] = y0 + (y1 - y0) * (xs - x0) / (x1 - x0)
    return np.clip(np rint(lut), 0, 255).astype(np.uint8)

def intensity_transform(image, breakpoints):
    return cv2.LUT(image, build_lut(breakpoints))

emma = cv2.imread(r"images\emma.jpeg", cv2.IMREAD_GRAYSCALE)
if emma is None:
    raise FileNotFoundError(r"Could not find images\emma.jpeg")

q1_curves = {
    "Fig. 1a (given)": [[0, 0], [50, 50], [50, 100], [150, 255], [150, 150], [255, 255]],
    "Custom 1": [[0, 0], [40, 20], [90, 120], [160, 220], [255, 255]],
    "Custom 2 (selected)": [[0, 0], [50, 70], [110, 170], [180, 235], [255, 255]],
    "Custom 3": [[0, 0], [40, 30], [90, 140], [160, 230], [255, 255]],
}

fig, axes = plt.subplots(2, 5, figsize=(16, 8), dpi=130)
axes[0, 0].imshow(emma, cmap="gray", vmin=0, vmax=255)
axes[0, 0].set_title("Original")
```

```

axes[0, 0].axis("off")
r = np.arange(256)
for ax, (name, bp) in zip(axes[0, 1:], q1_curves.items()):
    ax.imshow(intensity_transform(emma, bp), cmap="gray", vmin=0, vmax=255)
    ax.set_title(name)
    ax.axis("off")
for name, bp in q1_curves.items():
    axes[1, 0].plot(r, build_lut(bp), label=name)
axes[1, 0].plot(r, r, "k:", label="Identity")
axes[1, 0].set(title="Compared transformation curves", xlabel="Input intensity",
axes[1, 0].grid(alpha=.3); axes[1, 0].legend(fontsize=7)
for ax in axes[1, 1:]: ax.axis("off")
plt.tight_layout(); plt.show()

```



Task 2 - Accentuate white matter and gray matter

Brief description: Examine brain-image intensities and use different piecewise-linear mappings to emphasize bright white matter and the mid-intensity gray-matter band.

Chosen mappings: The selected white-matter mapping emphasizes intensities above about 150. The selected gray-matter mapping expands the approximately 120–150 range and attenuates higher intensities, following the reference approach.

```

In [2]: brain = cv2.imread(r"images\brain.jpeg", cv2.IMREAD_GRAYSCALE)
if brain is None:
    raise FileNotFoundError(r"Could not find images\brain.jpeg")

white_bp = [[0,0], [50,20], [150,80], [200,230], [255,255]]
gray_bp  = [[0,0], [120,180], [150,255], [230,100], [255,150]]

white = intensity_transform(brain, white_bp)
gray  = intensity_transform(brain, gray_bp)
r = np.arange(256)

fig, ax = plt.subplots(2, 3, figsize=(14,8), dpi=130)

# Images
ax[0,0].imshow(brain, cmap="gray", vmin=0, vmax=255)
ax[0,0].set_title("Original"); ax[0,0].axis("off")

```

```

ax[0,1].imshow(white, cmap="gray", vmin=0, vmax=255)
ax[0,1].set_title("White-matter enhancement"); ax[0,1].axis("off")

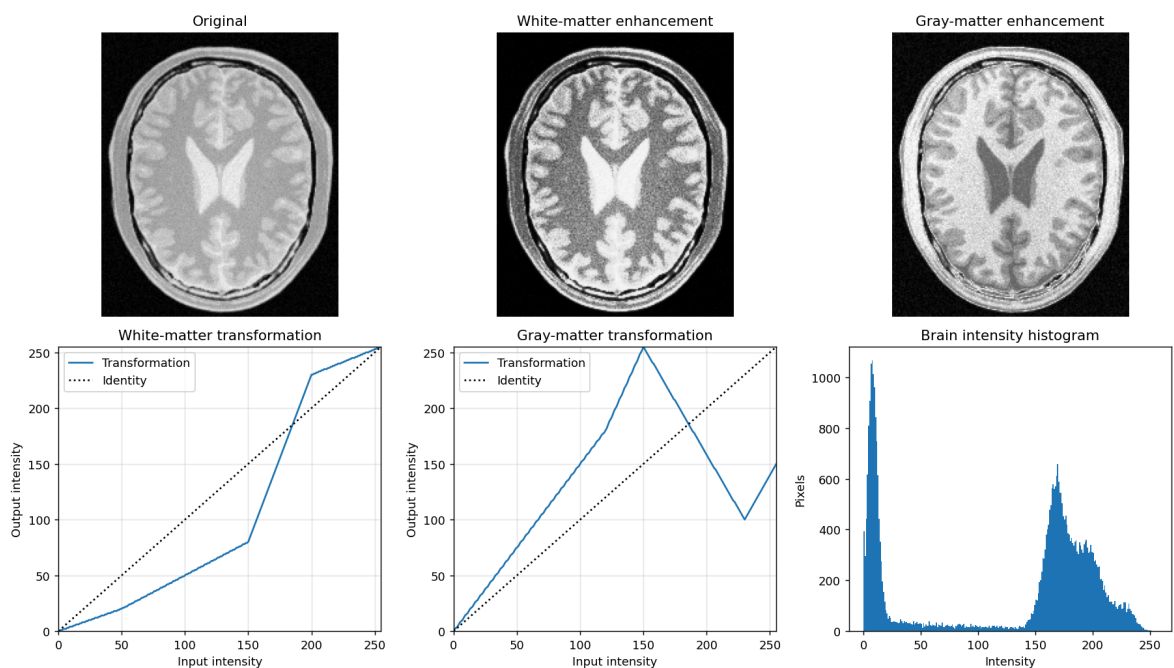
ax[0,2].imshow(gray, cmap="gray", vmin=0, vmax=255)
ax[0,2].set_title("Gray-matter enhancement"); ax[0,2].axis("off")

# Transformation curves
for a, bp, title in zip(ax[1,:2], [white_bp, gray_bp],
                        ["White-matter transformation", "Gray-matter transforma
a.plot(r, build_lut(bp), label="Transformation")
a.plot(r, r, "k:", label="Identity")
a.set(title=title, xlabel="Input intensity", ylabel="Output intensity",
      xlim=(0,255), ylim=(0,255))
a.grid(alpha=.3); a.legend()

# Histogram
ax[1,2].hist(brain.ravel(), bins=256, range=(0,256))
ax[1,2].set(title="Brain intensity histogram",
           xlabel="Intensity", ylabel="Pixels")

plt.tight_layout()
plt.show()

```



Task 3 - Gamma correction in CIE L,a,b space

Brief description: Brighten or darken the image by applying gamma correction only to the L* (lightness) channel. Keeping a* and b* unchanged preserves the colour information.

Tested values: $\gamma = 0.4, 0.6, 0.8, 1.0, 1.2$. **Chosen value:** $\gamma = 0.6$; it reveals dark-region detail without looking over-bright.

```

In [3]: image3 = cv2.imread(r"images\figure3.jpeg")
if image3 is None: raise FileNotFoundError("Could not find images\figure3.jpeg")

def gamma_correct_lab(image_bgr, gamma):

```

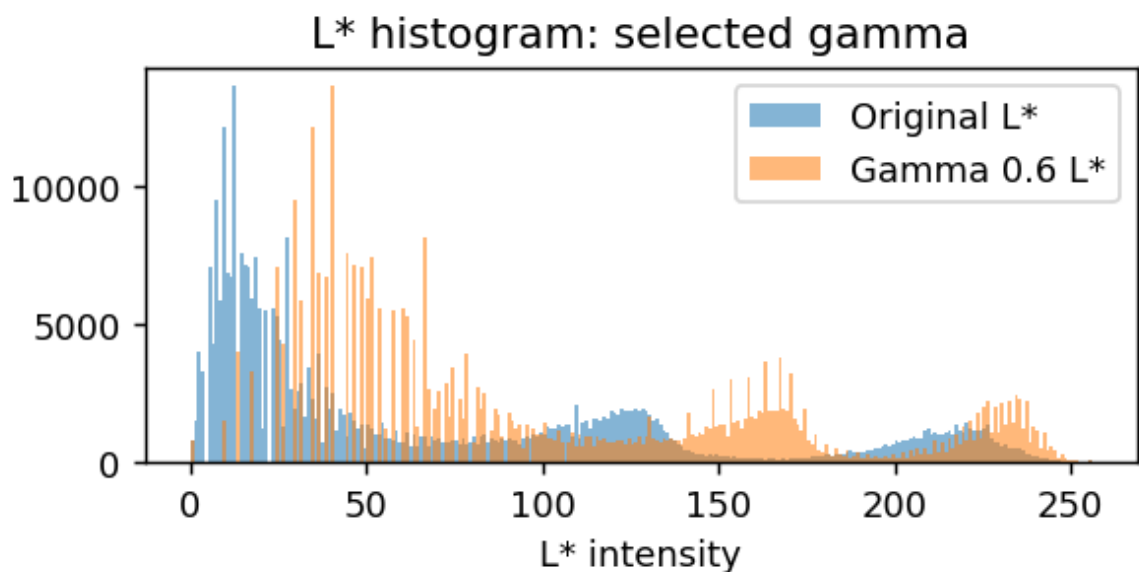
```

lab = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(lab)
corrected_L = np.clip(255 * (L.astype(np.float32) / 255) ** gamma, 0, 255).a
corrected = cv2.cvtColor(cv2.merge([corrected_L, a, b]), cv2.COLOR_LAB2RGB)
return corrected, L, corrected_L

gammas = [0.4, 0.6, 0.8, 1.0, 1.2]
fig, axes = plt.subplots(2, 3, figsize=(13, 8), dpi=130)
axes[0, 0].imshow(cv2.cvtColor(image3, cv2.COLOR_BGR2RGB)); axes[0, 0].set_title
x = np.arange(256)
for ax, gamma in zip(axes.flat[1:], gammas):
    corrected, _, _ = gamma_correct_lab(image3, gamma)
    ax.imshow(corrected); ax.set_title(f"Gamma = {gamma}" + (" (selected)" if ga
plt.tight_layout(); plt.show()

final3, L_before, L_after = gamma_correct_lab(image3, .6)
plt.figure(figsize=(5, 2), dpi=130); plt.hist(L_before.ravel(), 256, (0,256), al

```



Task 4 - Vibrance enhancement

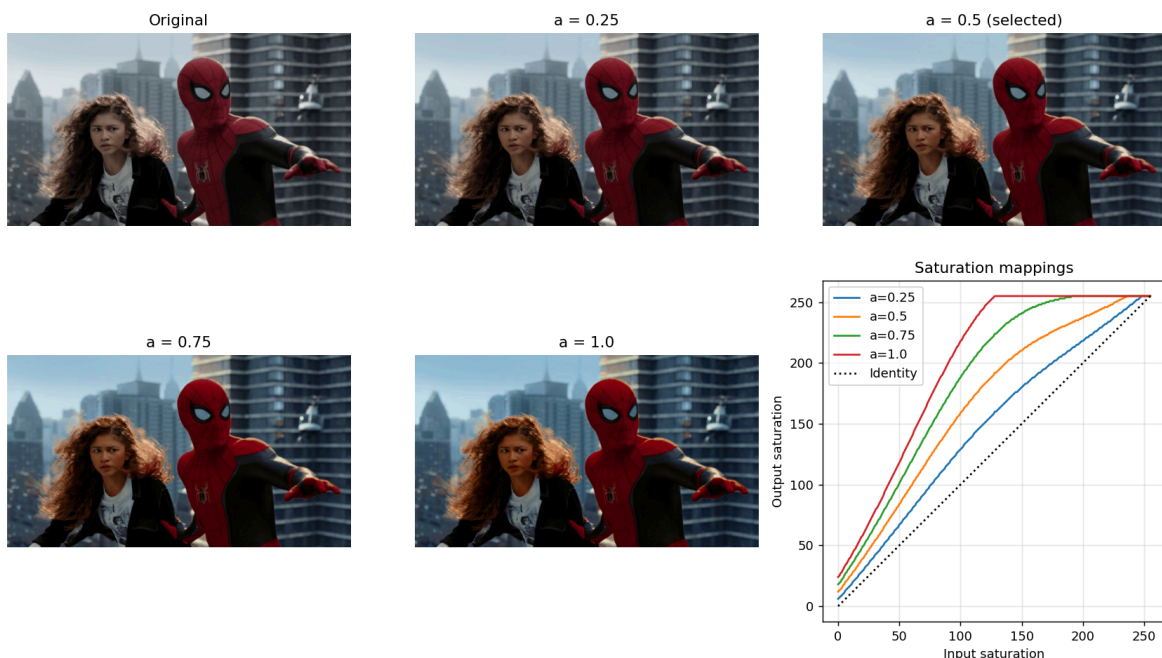
Brief description: Enhance colour vibrance by transforming only the HSV saturation channel. The Gaussian-shaped increase mostly affects intermediate saturation values while preserving hue and value.

Tested values: $a = 0.25, 0.50, 0.75, 1.00$ with $\sigma = 70$. **Chosen parameters:** $a = 0.50$ and $\sigma = 70$, giving clear colour enhancement without oversaturation.

```
In [4]: image4 = cv2.imread(r"images\figure4.jpeg")
if image4 is None: raise FileNotFoundError("Could not find images\figure4.jpeg")
hsv4 = cv2.cvtColor(image4, cv2.COLOR_BGR2HSV)
H4, S4, V4 = cv2.split(hsv4)

def vibrance_transform(saturation, a, sigma=70):
    values = saturation.astype(np.float32)
    result = values + a * 128 * np.exp(-((values - 128) ** 2) / (2 * sigma ** 2))
    return np.clip(result, 0, 255).astype(np.uint8)

a_values, sigma = [.25, .50, .75, 1.00], 70
fig, axes = plt.subplots(2, 3, figsize=(13, 8), dpi=130)
axes[0, 0].imshow(cv2.cvtColor(image4, cv2.COLOR_BGR2RGB)); axes[0, 0].set_title
for ax, a in zip(axes.flat[1:5], a_values):
    s_new = vibrance_transform(S4, a, sigma)
    result = cv2.cvtColor(cv2.merge([H4, s_new, V4]), cv2.COLOR_HSV2RGB)
    ax.imshow(result); ax.set_title(f"a = {a}" + (" (selected)" if a == .5 else
x = np.arange(256, dtype=np.float32)
for a in a_values: axes[1, 2].plot(x, vibrance_transform(x, a, sigma), label=f"a
axes[1, 2].plot(x, x, "k:", label="Identity"); axes[1, 2].set(title="Saturation
plt.tight_layout(); plt.show()
```

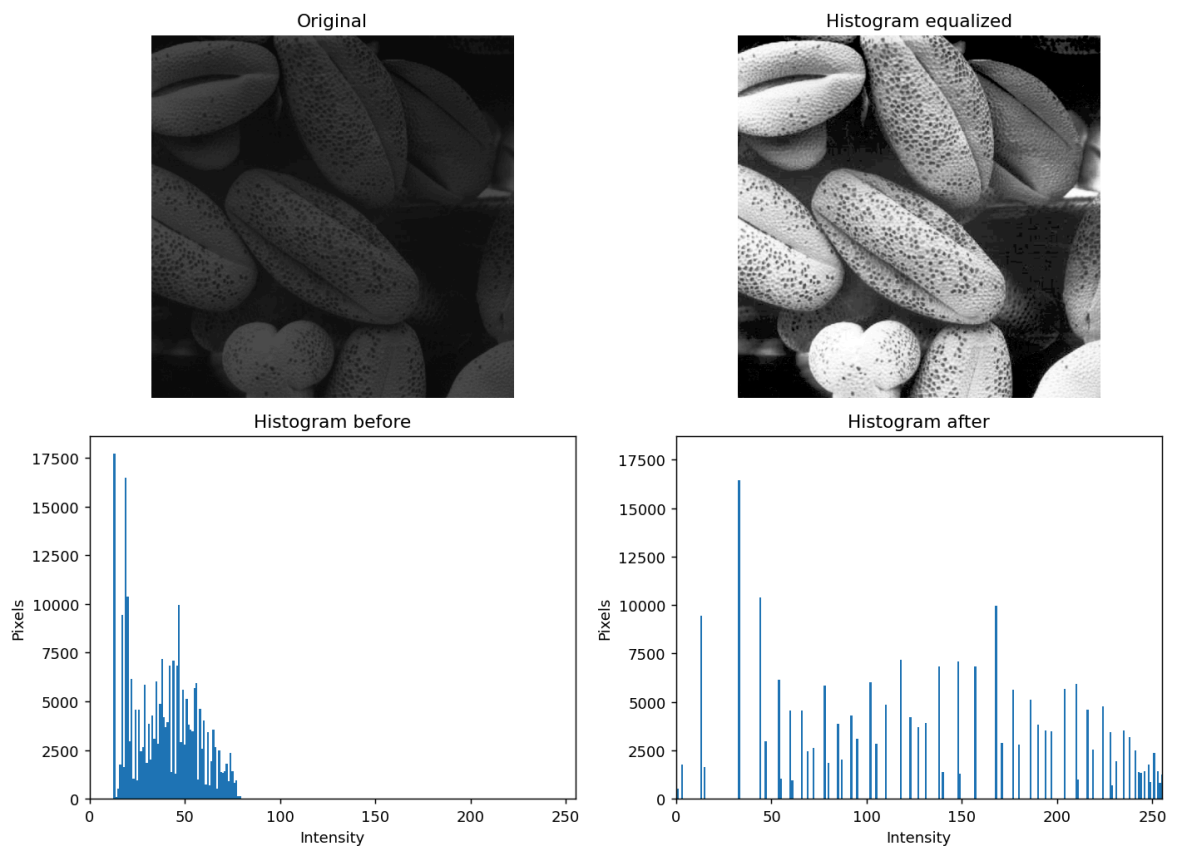


Task 5 - Histogram equalization

Brief description: Implement grayscale histogram equalization using a cumulative distribution function (CDF), then compare the image and histogram before and after equalization. The transformation redistributes available intensity levels and improves contrast.

```
In [5]: def histogram_equalization(image):
hist = np.bincount(image.ravel(), minlength=256)
cdf = np.cumsum(hist)
cdf_min = cdf[cdf > 0][0]
lut = np.round(255 * (cdf - cdf_min) / (image.size - cdf_min))
lut = np.clip(lut, 0, 255).astype(np.uint8)
return lut[image], hist, lut

shells = cv2.imread(r"images\shells.tif", cv2.IMREAD_GRAYSCALE)
if shells is None: raise FileNotFoundError(r"Could not find images\shells.tif")
shells_eq, hist_before, _ = histogram_equalization(shells)
hist_after = np.bincount(shells_eq.ravel(), minlength=256)
fig, axes = plt.subplots(2, 2, figsize=(11, 8), dpi=130)
axes[0, 0].imshow(shells, cmap="gray", vmin=0, vmax=255); axes[0, 0].set_title("
axes[0, 1].imshow(shells_eq, cmap="gray", vmin=0, vmax=255); axes[0, 1].set_titl
axes[1, 0].bar(np.arange(256), hist_before, width=1); axes[1, 0].set_title("Hist
axes[1, 1].bar(np.arange(256), hist_after, width=1); axes[1, 1].set_title("Histo
for ax in axes[1]: ax.set(xlabel="Intensity", ylabel="Pixels", xlim=(0,255))
plt.tight_layout(); plt.show()
```



Task 6 - Foreground-only histogram equalization

Brief description: Separate the foreground from the low-saturation background using HSV saturation, build a foreground mask, and equalize only the foreground value (V) channel. This keeps the background and the foreground hue/saturation unchanged.

(a) Inspect HSV planes

Convert the colour image to HSV and display the hue, saturation, and value planes in grayscale. This helps identify the channel that best separates the foreground from the

background.

```
In [6]: image6 = cv2.imread(r"images\figure6.jpeg")
if image6 is None:
    raise FileNotFoundError(r"Could not find images\figure6.jpeg")

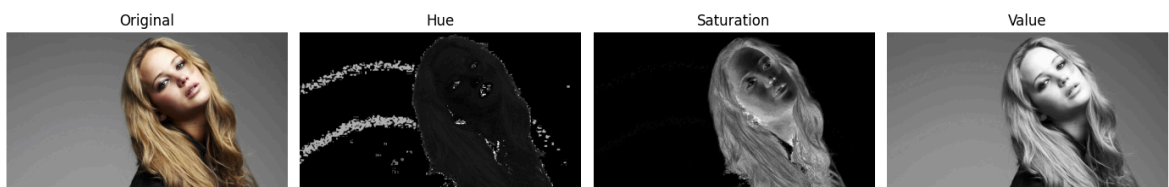
hsv6 = cv2.cvtColor(image6, cv2.COLOR_BGR2HSV)
H6, S6, V6 = cv2.split(hsv6)

fig, ax = plt.subplots(1, 4, figsize=(14, 4))

ax[0].imshow(cv2.cvtColor(image6, cv2.COLOR_BGR2RGB))
ax[0].set_title("Original"); ax[0].axis("off")

for a, plane, title in zip(ax[1:], [H6, S6, V6], ["Hue", "Saturation", "Value"]):
    a.imshow(plane, cmap="gray")
    a.set_title(title); a.axis("off")

plt.tight_layout()
plt.show()
```



(b) Create the foreground mask

Threshold the saturation plane using Otsu's method to create a binary mask. Keep and fill the largest external contour to obtain the complete main foreground object.

```
In [7]: # Threshold the saturation channel to obtain foreground mask
threshold, mask = cv2.threshold(
    S6, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
print(f"Otsu threshold: {threshold:.0f}")

# Find the largest external contour
contours, _ = cv2.findContours(
    mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)
largest_contour = max(contours, key=cv2.contourArea)

# Fill the largest foreground contour
mask_filled = np.zeros_like(mask)
cv2.drawContours(
    mask_filled, [largest_contour], -1, 255, thickness=cv2.FILLED
)

fig, axes = plt.subplots(1, 2, figsize=(8, 4), dpi=130)

axes[0].imshow(mask, cmap="gray")
axes[0].set_title("Thresholded Foreground Mask")
axes[0].axis("off")

axes[1].imshow(mask_filled, cmap="gray")
```

```
axes[1].set_title("Filled Foreground Mask")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```

Otsu threshold: 60

Thresholded Foreground Mask



Filled Foreground Mask



(c) Obtain foreground and its histogram

Use `cv2.bitwise_and` with the binary mask to retain only foreground pixels from the V channel. Compute the histogram using only these foreground pixel values.

```
In [8]: # Extract foreground V channel
foreground_V = cv2.bitwise_and(V6, V6, mask=mask_filled)

# Foreground pixel values and histogram
fg_values = foreground_V[mask_filled > 0]
hist = np.bincount(fg_values, minlength=256)

fig, axes = plt.subplots(1, 2, figsize=(10, 4), dpi=130)

axes[0].imshow(foreground_V, cmap="gray", vmin=0, vmax=255)
axes[0].set_title("Foreground V")
axes[0].axis("off")

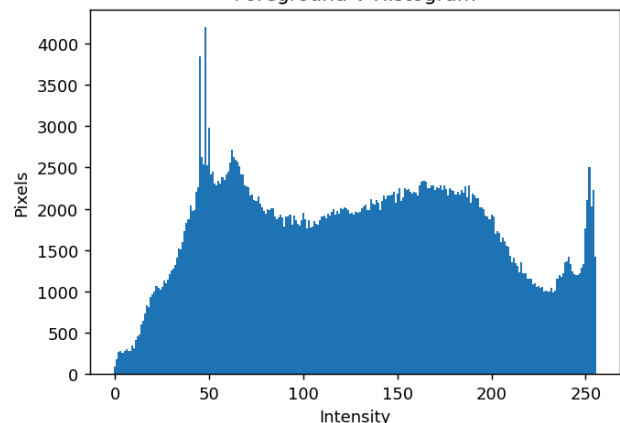
axes[1].bar(np.arange(256), hist, width=1)
axes[1].set_title("Foreground V Histogram")
axes[1].set_xlabel("Intensity")
axes[1].set_ylabel("Pixels")

plt.tight_layout()
plt.show()
```

Foreground V



Foreground V Histogram

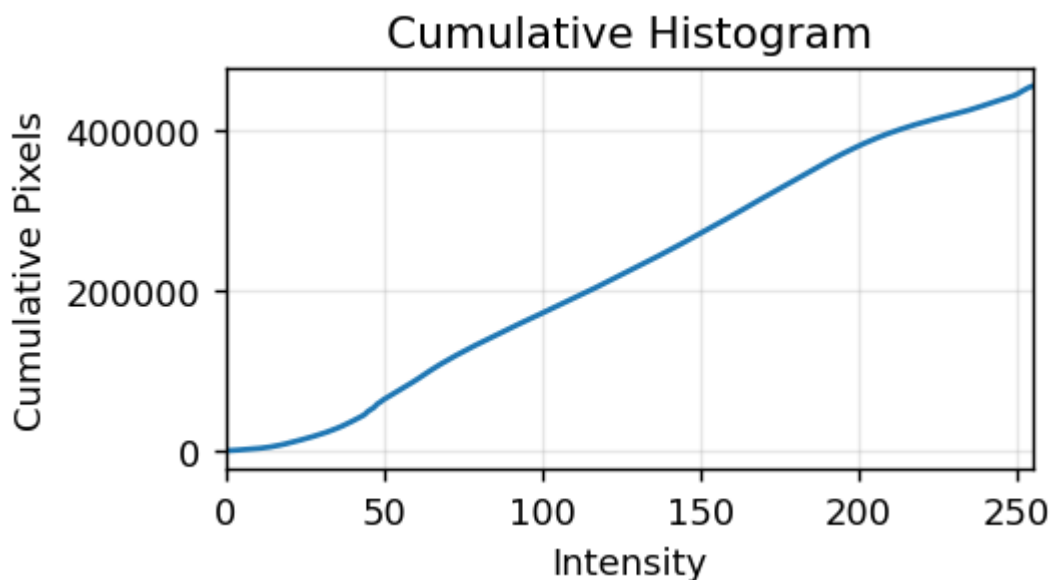


(d) Compute the cumulative histogram

Use `np.cumsum` to calculate the cumulative distribution function (CDF) of the foreground histogram. This CDF is required to construct the histogram-equalization mapping.

```
In [9]: # Compute cumulative histogram
cdf = np.cumsum(hist)

plt.figure(figsize=(4, 2), dpi=130)
plt.plot(cdf)
plt.title("Cumulative Histogram")
plt.xlabel("Intensity")
plt.ylabel("Cumulative Pixels")
plt.xlim(0, 255)
plt.grid(alpha=.3)
plt.show()
```



(e) Equalize the foreground

Create a lookup table from the foreground CDF and apply it only to pixels inside the foreground mask. Compare the foreground image and histogram before and after equalization.

```
In [10]: # Compute histogram-equalization LUT
cdf_nonzero = cdf[cdf > 0]
cdf_min = cdf_nonzero[0]
N = len(fg_values)

lut = np.round(
    (cdf - cdf_min) / (N - cdf_min) * 255
)
lut = np.clip(lut, 0, 255).astype(np.uint8)

# Equalize ONLY foreground pixels
V_equalized = V6.copy()
foreground_pixels = mask_filled > 0
```

```

V_equalized[foreground_pixels] = lut[V6[foreground_pixels]]

# Extract equalized foreground and histogram
foreground_V_equalized = cv2.bitwise_and(
    V_equalized, V_equalized, mask=mask_filled
)

fg_eq_values = foreground_V_equalized[mask_filled > 0]
hist_eq = np.bincount(fg_eq_values, minlength=256)

fig, axes = plt.subplots(2, 2, figsize=(10, 8), dpi=130)

axes[0, 0].imshow(foreground_V, cmap="gray", vmin=0, vmax=255)
axes[0, 0].set_title("Original Foreground V")
axes[0, 0].axis("off")

axes[0, 1].imshow(foreground_V_equalized, cmap="gray", vmin=0, vmax=255)
axes[0, 1].set_title("Equalized Foreground V")
axes[0, 1].axis("off")

axes[1, 0].bar(np.arange(256), hist, width=1)
axes[1, 0].set_title("Original Foreground Histogram")
axes[1, 0].set_xlabel("Intensity")
axes[1, 0].set_ylabel("Pixels")

axes[1, 1].bar(np.arange(256), hist_eq, width=1)
axes[1, 1].set_title("Equalized Foreground Histogram")
axes[1, 1].set_xlabel("Intensity")
axes[1, 1].set_ylabel("Pixels")

plt.tight_layout()
plt.show()

```

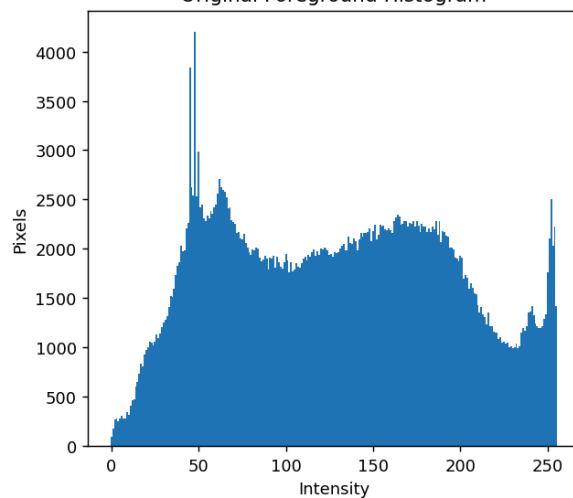
Original Foreground V



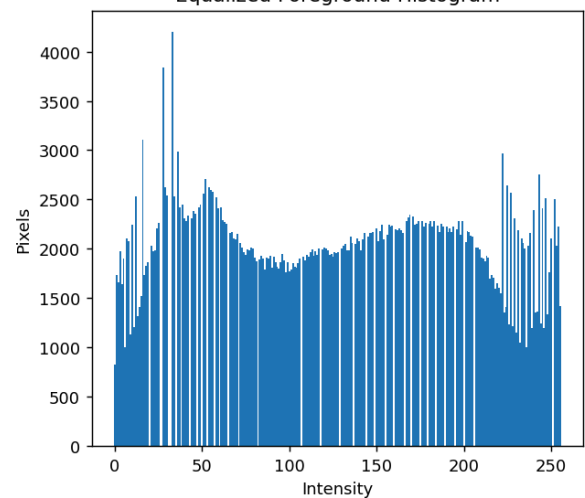
Equalized Foreground V



Original Foreground Histogram



Equalized Foreground Histogram



(f) Recombine and display the result

Merge the original hue and saturation channels with the equalized V channel, then convert the HSV result back to RGB. This improves foreground contrast while preserving the original colours and background.

```
In [11]: # Recombine original H and S with equalized V
hsv_equalized = cv2.merge([H6, S6, V_equalized])

# Convert HSV -> RGB for display
result_rgb = cv2.cvtColor(
    hsv_equalized, cv2.COLOR_HSV2RGB
)

fig, axes = plt.subplots(1, 2, figsize=(10, 4), dpi=130)

axes[0].imshow(cv2.cvtColor(image6, cv2.COLOR_BGR2RGB))
axes[0].set_title("Original Image")
axes[0].axis("off")

axes[1].imshow(result_rgb)
axes[1].set_title("Foreground Histogram Equalized")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Foreground Histogram Equalized



Q.7 - Sobel Filtering

The Sobel X filter mainly detects vertical edges because it measures intensity changes in the horizontal direction. Similarly, the Sobel Y filter mainly detects horizontal edges by measuring intensity changes in the vertical direction. The gradient magnitude combines both responses using

$$G = \sqrt{G_x^2 + G_y^2}$$

therefore highlighting edges in different orientations.

In Q.7.b, the Sobel filtering operation was implemented manually by padding the image and sliding the kernel over every pixel. The resulting gradient images were visually consistent with those obtained using `cv.filter2D`.

In Q.7.c, the Sobel kernels were implemented as separable filters. The (G_x) kernel was decomposed into a vertical smoothing filter and a horizontal derivative filter, while the (G_y) kernel was decomposed into a horizontal smoothing filter and a vertical derivative filter. The resulting images were consistent with the original 2-D Sobel filtering results.

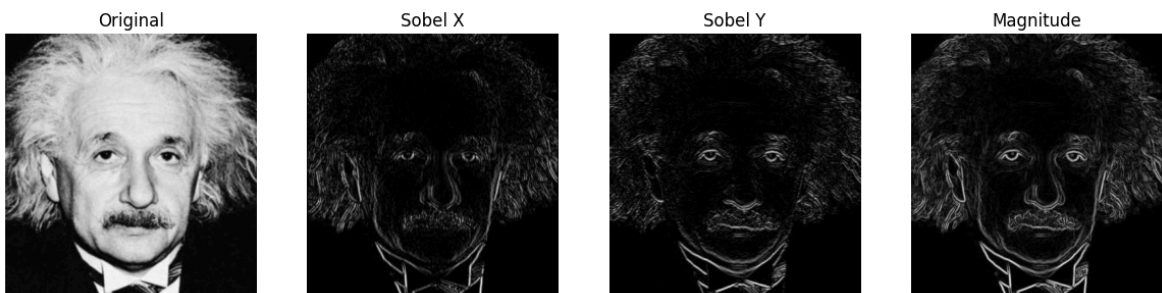
```
In [12]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img_q7 = cv.imread(r"images\figure7.jpeg", cv.IMREAD_GRAYSCALE)

sobel_x = np.array([[1,0,-1],[2,0,-2],[1,0,-1]], np.float32)
sobel_y = np.array([[1,2,1],[0,0,0],[-1,-2,-1]], np.float32)

Gx = cv.filter2D(img_q7.astype(np.float32), cv.CV_32F, sobel_x)
Gy = cv.filter2D(img_q7.astype(np.float32), cv.CV_32F, sobel_y)
mag = np.sqrt(Gx**2 + Gy**2)
mag = cv.normalize(mag, None, 0, 255, cv.NORM_MINMAX).astype(np.uint8)

fig, ax = plt.subplots(1, 4, figsize=(12, 3))
for a, im, t in zip(ax, [img_q7, abs(Gx), abs(Gy), mag],
                    ["Original", "Sobel X", "Sobel Y", "Magnitude"]):
    a.imshow(im, cmap="gray")
    a.set_title(t)
    a.axis("off")
plt.tight_layout()
plt.show()
```



```
In [13]: def filter2d_manual(image, kernel):
    kh, kw = kernel.shape
    ph, pw = kh//2, kw//2
    image = image.astype(np.float32)
    padded = np.pad(image, ((ph,ph),(pw,pw)), mode="reflect")
    output = np.zeros_like(image)

    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            output[i,j] = np.sum(padded[i:i+kh, j:j+kw] * kernel)
    return output

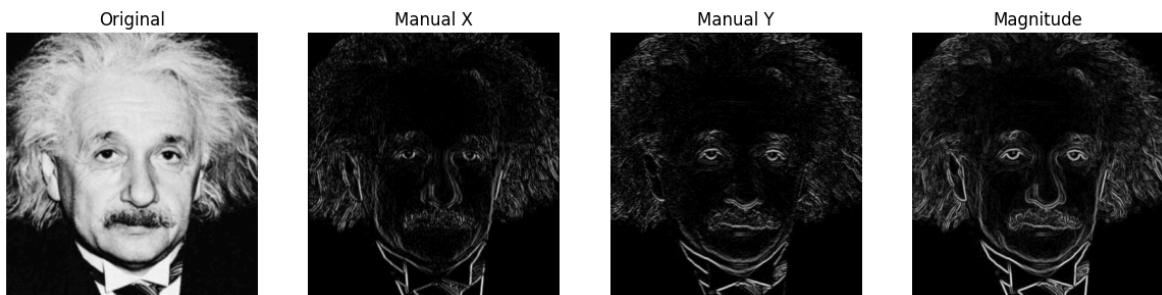
Gx_m = filter2d_manual(img_q7, sobel_x)
Gy_m = filter2d_manual(img_q7, sobel_y)
mag_m = cv.normalize(np.sqrt(Gx_m**2 + Gy_m**2),
                    None, 0, 255, cv.NORM_MINMAX).astype(np.uint8)

fig, ax = plt.subplots(1, 4, figsize=(12, 3))
for a, im, t in zip(ax, [img_q7, abs(Gx_m), abs(Gy_m), mag_m],
                    ["Original", "Manual X", "Manual Y", "Magnitude"]):
    a.imshow(im, cmap="gray")
```

```

a.set_title(t)
a.axis("off")
plt.tight_layout()
plt.show()

```



```

In [14]: smooth_col = np.array([[1],[2],[1]], np.float32)
diff_row = np.array([[1,0,-1]], np.float32)
smooth_row = np.array([[1,2,1]], np.float32)
diff_col = np.array([[1],[0],[-1]], np.float32)

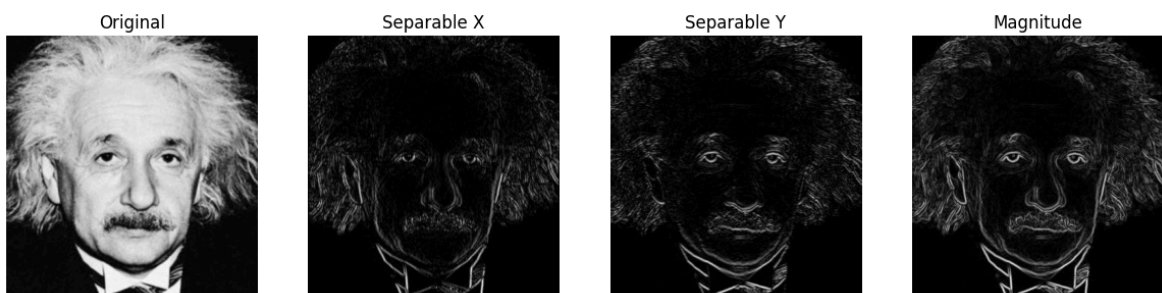
img = img_q7.astype(np.float32)

Gx_s = cv.filter2D(cv.filter2D(img, cv.CV_32F, smooth_col),
                  cv.CV_32F, diff_row)
Gy_s = cv.filter2D(cv.filter2D(img, cv.CV_32F, smooth_row),
                  cv.CV_32F, diff_col)

mag_s = cv.normalize(np.sqrt(Gx_s**2 + Gy_s**2),
                    None, 0, 255, cv.NORM_MINMAX).astype(np.uint8)

fig, ax = plt.subplots(1, 4, figsize=(12, 3))
for a, im, t in zip(ax, [img_q7, abs(Gx_s), abs(Gy_s), mag_s],
                    ["Original", "Separable X", "Separable Y", "Magnitude"]):
    a.imshow(im, cmap="gray")
    a.set_title(t)
    a.axis("off")
plt.tight_layout()
plt.show()

```



Q.8 Image Zooming

The image was zoomed by a factor of 4 using nearest-neighbor and bilinear interpolation.

Nearest-neighbor interpolation assigns each output pixel the value of the closest source pixel, which produces block-like artifacts after zooming. Bilinear interpolation estimates the pixel value using the four nearest source pixels, resulting in smoother transitions.

The normalized sum of squared difference (NSSD) was calculated as

$$\text{NSSD} = \frac{\sum (I_{\text{original}} - I_{\text{zoomed}})^2}{\sum I_{\text{original}}^2}.$$

The bilinear interpolation generally produced a lower NSSD than nearest-neighbor interpolation, indicating that it was closer to the corresponding original image. However, zooming cannot recover fine details that were already lost in the smaller images.

```
In [15]: def zoom_image(image, scale, method="nearest"):
    if not 0 < scale <= 10:
        raise ValueError("Scale must be in (0, 10].")

    h, w = image.shape[:2]
    new_h, new_w = round(h * scale), round(w * scale)
    img = image.astype(np.float32)
    output = np.zeros((new_h, new_w, *image.shape[2:]), np.float32)

    for y in range(new_h):
        for x in range(new_w):
            xs = (x + 0.5) / scale - 0.5
            ys = (y + 0.5) / scale - 0.5

            if method == "nearest":
                xi = np.clip(round(xs), 0, w - 1)
                yi = np.clip(round(ys), 0, h - 1)
                output[y, x] = img[yi, xi]

            elif method == "bilinear":
                xs, ys = np.clip(xs, 0, w - 1), np.clip(ys, 0, h - 1)
                x1, y1 = int(np.floor(xs)), int(np.floor(ys))
                x2, y2 = min(x1 + 1, w - 1), min(y1 + 1, h - 1)
                dx, dy = xs - x1, ys - y1

                output[y, x] = (
                    (1-dx)*(1-dy)*img[y1,x1] +
                    dx*(1-dy)*img[y1,x2] +
                    (1-dx)*dy*img[y2,x1] +
                    dx*dy*img[y2,x2]
                )

            else:
                raise ValueError("method must be 'nearest' or 'bilinear'")

    return np.clip(output, 0, 255).astype(np.uint8)
```

```
In [16]: image_pairs = [
    ("images/im01small.png", "images/im01.png", "Image 01"),
    ("images/im02small.png", "images/im02.png", "Image 02"),
    ("images/im03small.png", "images/im03.png", "Image 03"),
    ("images/im04small.jpg", "images/im04.jpg", "Image 04")
]

results = []

for small_path, original_path, name in image_pairs:
    small = cv.cvtColor(cv.imread(small_path), cv.COLOR_BGR2RGB)
```

```

original = cv.cvtColor(cv.imread(original_path), cv.COLOR_BGR2RGB)

nearest = zoom_image(small, 4, "nearest")
bilinear = zoom_image(small, 4, "bilinear")

h = min(original.shape[0], nearest.shape[0], bilinear.shape[0])
w = min(original.shape[1], nearest.shape[1], bilinear.shape[1])

original = original[:h, :w]
nearest = nearest[:h, :w]
bilinear = bilinear[:h, :w]

# NSSD
I = original.astype(np.float64)
N = nearest.astype(np.float64)
B = bilinear.astype(np.float64)

nssd_nearest = np.sum((I-N)**2) / np.sum(I**2)
nssd_bilinear = np.sum((I-B)**2) / np.sum(I**2)

results.append([name, nssd_nearest, nssd_bilinear])

# Central crop
ch, cw = int(h*0.25), int(w*0.25)
cy, cx = h//2, w//2

y1, y2 = cy-ch//2, cy+ch//2
x1, x2 = cx-cw//2, cx+cw//2

crops = [
    original[y1:y2, x1:x2],
    nearest[y1:y2, x1:x2],
    bilinear[y1:y2, x1:x2]
]

fig, ax = plt.subplots(1, 3, figsize=(15, 5))

for a, im, title in zip(
    ax, crops, ["Original", "Nearest Neighbor", "Bilinear"]
):
    a.imshow(im)
    a.set_title(title)
    a.axis("off")

fig.suptitle(name)
plt.tight_layout()
plt.show()

print("NSSD Results")
for name, n, b in results:
    print(f"{name}: Nearest = {n:.6f}, Bilinear = {b:.6f}")

```

Image 01



Image 02

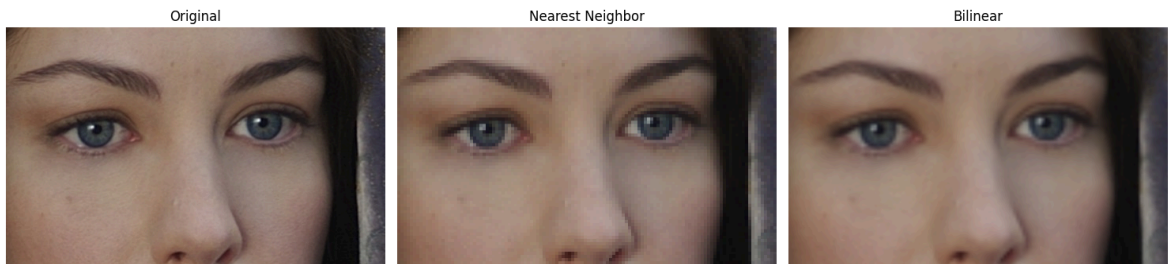


Image 03

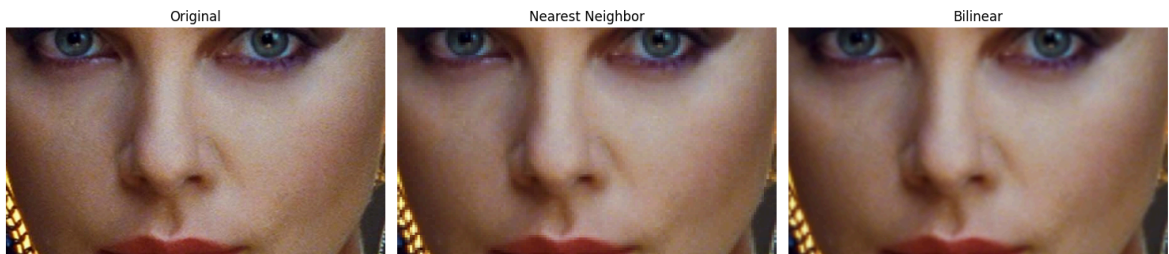
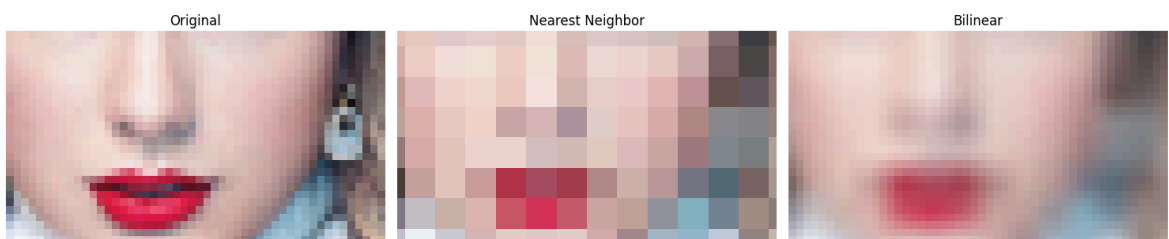


Image 04



NSSD Results

Image 01: Nearest = 0.012058, Bilinear = 0.010187

Image 02: Nearest = 0.004190, Bilinear = 0.002915

Image 03: Nearest = 0.007668, Bilinear = 0.005823

Image 04: Nearest = 0.005354, Bilinear = 0.004608

Q9 - GrabCut and Background Blur

```
In [17]: img = cv.imread("images/figure8.jpeg")
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
h, w = img.shape[:2]

mask = np.zeros((h, w), np.uint8)
bg_model = np.zeros((1, 65), np.float64)
fg_model = np.zeros((1, 65), np.float64)

rect = (int(.05*w), int(.12*h), int(.95*w), int(.6*h))

# Show GrabCut rectangle
```

```

rect_img = img_rgb.copy()
x, y, rw, rh = rect
cv.rectangle(rect_img, (x, y), (x+rw, y+rh), (255, 0, 0), 3)

cv.grabCut(img, mask, rect, bg_model, fg_model, 10,
           cv.GC_INIT_WITH_RECT)

mask = np.where(
    (mask == cv.GC_FGD) | (mask == cv.GC_PR_FGD), 1, 0
).astype(np.uint8)

# Keep Largest foreground component
n, labels, stats, _ = cv.connectedComponentsWithStats(mask)
largest = 1 + np.argmax(stats[1:, cv.CC_STAT_AREA])
mask = (labels == largest).astype(np.uint8)

foreground = img_rgb * mask[:, :, None]
background = img_rgb * (1 - mask[:, :, None])

fig, ax = plt.subplots(1, 5, figsize=(17, 4))

for a, im, title in zip(
    ax,
    [img_rgb, rect_img, mask, foreground, background],
    ["Original", "GrabCut Rectangle", "Segmentation Mask",
     "Foreground", "Background"]
):
    a.imshow(im, cmap="gray" if title == "Segmentation Mask" else None)
    a.set_title(title)
    a.axis("off")

plt.tight_layout()
plt.show()

```



```

In [18]: # Blur original image first
blurred = cv.GaussianBlur(background, (31, 31), 10)

# Keep foreground sharp and background blurred
enhanced = np.where(
    mask[:, :, None] == 1,
    img_rgb,
    blurred
).astype(np.uint8)

fig, ax = plt.subplots(1, 2, figsize=(10, 5))
for a, im, title in zip(
    ax, [img_rgb, enhanced], ["Original", "Background Blurred"]
):
    a.imshow(im)
    a.set_title(title)
    a.axis("off")

```

```
plt.tight_layout()
plt.show()
```

Original



Background Blurred



Q.9(c)

The dark region around the flower occurs when the extracted background image is blurred after the foreground pixels have been replaced by zeros. Near the segmentation boundary, the Gaussian filter averages normal background pixels with these zero-valued pixels. This reduces the intensity and creates a dark halo around the flower.

Therefore, the background should preferably be blurred **before** applying the segmentation mask, as done in Q.9(b).

Q.10 - Gaussian and Bilateral Filtering

Gaussian filtering smooths an image using spatial distance only. As a result, pixels across an edge can contribute to the smoothing and cause edge blurring.

Bilateral filtering considers both spatial distance and intensity similarity. Pixels that are far away or have significantly different intensities receive smaller weights. Therefore, relatively uniform regions are smoothed while strong edges are better preserved.

```
In [19]: img = cv.imread("images/figure9.jpeg")
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)

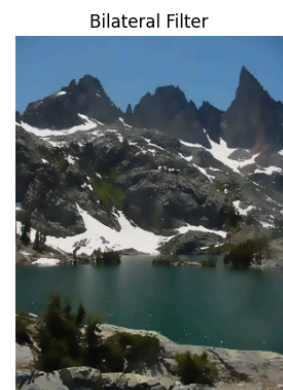
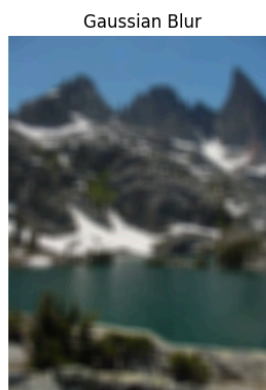
d = 11
sigma_s = 15
sigma_r = 60

gaussian = cv.GaussianBlur(img, (d, d), sigma_s)
bilateral = cv.bilateralFilter(img, d, sigma_r, sigma_s)
```

```
fig, ax = plt.subplots(1, 3, figsize=(14, 4))

for a, im, title in zip(
    ax,
    [img_rgb,
     cv.cvtColor(gaussian, cv.COLOR_BGR2RGB),
     cv.cvtColor(bilateral, cv.COLOR_BGR2RGB)],
    ["Original", "Gaussian Blur", "Bilateral Filter"]
):
    a.imshow(im)
    a.set_title(title)
    a.axis("off")

plt.tight_layout()
plt.show()
```



```
In [20]: def bilateral_manual(image, d, sigma_s, sigma_r):
    img = image.astype(np.float32)
    r = d // 2
    padded = np.pad(img, ((r,r),(r,r),(0,0)), mode="reflect")
    out = np.zeros_like(img)

    y, x = np.mgrid[-r:r+1, -r:r+1]
    spatial = np.exp(-(x**2 + y**2)/(2*sigma_s**2))

    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            patch = padded[i:i+d, j:j+d]
            diff = np.sum((patch - img[i,j])**2, axis=2)
            range_w = np.exp(-diff/(2*sigma_r**2))
            weights = spatial * range_w

            out[i,j] = np.sum(
                patch * weights[:, :, None], axis=(0,1)
            ) / np.sum(weights)

    return np.clip(out, 0, 255).astype(np.uint8)

manual = bilateral_manual(img, d, sigma_s, sigma_r)
```

```
In [21]: fig, ax = plt.subplots(1, 4, figsize=(16, 4))

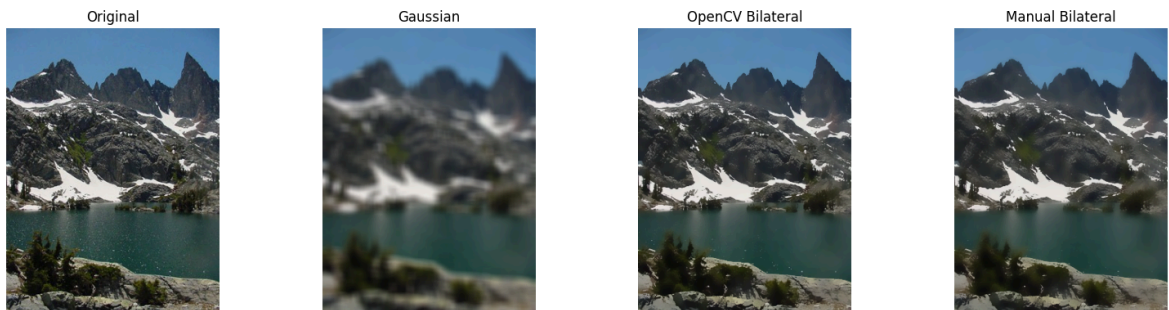
for a, im, title in zip(
    ax,
    [img_rgb,
     cv.cvtColor(gaussian, cv.COLOR_BGR2RGB),
     cv.cvtColor(bilateral, cv.COLOR_BGR2RGB),
```

```

    cv.cvtColor(manual, cv.COLOR_BGR2RGB)],
    ["Original", "Gaussian", "OpenCV Bilateral", "Manual Bilateral"]
):
    a.imshow(im)
    a.set_title(title)
    a.axis("off")

plt.tight_layout()
plt.show()

```



The manual bilateral filter produced results similar to OpenCV's implementation. The similarity was evaluated using MSE and PSNR. A lower MSE and higher PSNR indicate closer agreement between the two implementations. Small differences can occur due to implementation and border-handling details.

```

In [22]: mse = np.mean((manual.astype(float) - bilateral.astype(float))**2)
          psnr = 10 * np.log10(255**2 / mse)

          print(f"MSE = {mse:.4f}")
          print(f"PSNR = {psnr:.2f} dB")

```

```

MSE = 23.0056
PSNR = 34.51 dB

```